

A Project Report on
“GPS CLOCK SYSTEM USING LAUNCHXLTMS320F28379D”

*A Project Report submitted in the partial fulfillment of the requirements for the award
of the*

INTERNSHIP

IN

**MEDHA SERVO DRIVES PVT
LTD.**

Submitted By

PAMERA SUMITH KUMAR



**Under the Esteemed Guidance of
K. Satyanarayana, Sr.Manager**

MEDHA SERVO DRIVES PVT LTD R&D

Beside Government Mint Compound, Cherlapally, Hyderabad-508002

Telangana

2026

TABLE OF CONTENTS

Contents		Page No.
List of Figures		1
List of Tables		2 3
Abstract		
Chapter-1	Introduction	4-5
1.1	Introduction	
1.2	Aim of the project	
Chapter-2	Literature Survey	4-9
2.1	Research Papers	
Chapter-3	Hardware Description	10-17
3.1	Block Diagram Description	
3.2	Flow chart	
Chapter-4	Software Description	18-41
4.1	System Requirements	
4.2	System Environment	
4.3	Embedded C Language	
4.4	Advantages of Embedded C	
4.5	Features Of Embedded C	
4.6	Docklight	
4.7	NMEA Sentence Format	
4.9	GPS Clock System Code	
Chapter-5	Advantages And Applications	42-45

5.1	Advantages	
5.2	Applications	
Chapter-6	Results	46-48
6.1	Results	
6.2	Conclusion	
Chapter-7	References	49-50
7.1	References	

List of Figures

Fig no	Fig name	Page no
1	LaunchXL Pin Diagram	13
2	OLED Display	15
3	U-BLOX	15
4	Code Composer Studio	21
5	Docklight	26
6	OLED Display(time&date)	47
7	Docklight(NMEA)	47

List of Tables

Table No	Table name	Page No
1.	NMEA Sentence Format	27
2.	Talker ID	27

ABSTRACT

A GPS Clock System is designed to provide highly accurate time information by utilizing signals received from GPS satellites. In this project, a **GPS module**, **Texas Instruments LaunchXL development board**, and an **OLED display** are integrated to develop a real-time clock system with high precision.

The GPS module receives time and location data from satellites in the form of NMEA sentences. The LaunchXL pad processes this data by extracting precise time information provided by the GPS system, which is synchronized with atomic clocks. This processed time data is then displayed on the OLED screen in a user-friendly format.

The system eliminates the need for manual time setting and periodic calibration, as the GPS continuously updates the time automatically. The OLED display ensures low power consumption and clear visibility of time information. This GPS-based clock system is especially useful in applications requiring accurate time synchronization, such as communication systems, navigation devices, and embedded systems.

Overall, the proposed system demonstrates a reliable, low-cost, and accurate timekeeping solution using GPS technology and embedded hardware.

CHAPTER-1

INTRODUCTION

1.1 INTRODUCTION:

Accurate timekeeping plays a vital role in modern embedded systems, communication networks, navigation systems, and data synchronization applications. Conventional digital clocks and Real-Time Clock (RTC) modules often suffer from time drift and require manual adjustment or periodic calibration. To overcome these limitations, Global Positioning System (GPS) technology can be used to obtain highly precise time information.

GPS satellites are equipped with atomic clocks and transmit accurate time data along with location information. By receiving these signals through a GPS module, it is possible to build a clock system that provides extremely accurate and reliable time without manual intervention. This project focuses on designing a **GPS-based clock system** using a **GPS module**, **Texas Instruments LaunchXL development board**, and an **OLED display**.

The LaunchXL pad acts as the central processing unit, extracting time data from GPS NMEA sentences and controlling the display. The OLED display is used to show the current time clearly with low power consumption. This system ensures automatic time synchronization and high accuracy, making it suitable for real-time and mission-critical applications.

1.2 Aim Of The Project:

The primary aim of this project is to design and develop a **GPS-based accurate clock system** using a **GPS module**, **Texas Instruments LaunchXL development board**, and an **OLED display**. The project

focuses on obtaining precise real-time clock information directly from GPS satellites, which are synchronized with atomic clocks, thereby ensuring high accuracy and reliability.

Conventional clock systems and Real-Time Clock (RTC) modules often suffer from time drift and require manual adjustment or periodic calibration. This project aims to overcome these limitations by utilizing GPS technology to provide automatic time synchronization without human intervention. The LaunchXL development board acts as the central processing unit, receiving NMEA data from the GPS module through serial communication, extracting the required time information, and formatting it for display.

Another important aim of the project is to demonstrate effective hardware and software integration in an embedded system environment, including UART communication, data parsing, and OLED interfacing. The use of an OLED display ensures low power consumption, high contrast, and clear visibility of time information.

Overall, the project aims to provide a **low-cost, efficient, and reliable timekeeping solution** suitable for embedded systems, communication networks, navigation devices, and applications where precise time synchronization is essential.

CHAPTER-2
LITERATURE SURVEY

2.1 Research papers:

Accurate time synchronization is a critical requirement in many embedded and communication systems. Over the years, various methods and technologies have been developed to achieve precise timekeeping. This literature survey reviews existing approaches related to clock systems, GPS-based time synchronization, and display techniques used in embedded applications.

Traditional digital clock systems commonly use Real-Time Clock (RTC) modules such as DS1307 or DS3231. These RTC-based systems provide basic timekeeping functionality but suffer from time drift due to crystal inaccuracies and environmental factors. Several studies report that RTC modules require periodic manual correction to maintain accuracy, which limits their reliability in long-term applications.

With advancements in satellite navigation systems, GPS-based time synchronization has gained significant attention. GPS satellites are equipped with highly accurate atomic clocks and transmit precise time information to receivers. Many researchers have demonstrated that GPS can provide time accuracy within nanoseconds, making it superior to conventional RTC-based systems. GPS timing has been widely used in applications such as telecommunication networks, power grid synchronization, and navigation systems.

Several embedded system implementations have been proposed using GPS modules interfaced with microcontrollers through serial communication (UART). These systems extract time information from standard NMEA sentences such as GPRMC and GPWGA. Literature

shows that GPS-based clocks eliminate manual time setting and automatically adjust time based on satellite signals, thereby improving reliability and accuracy.

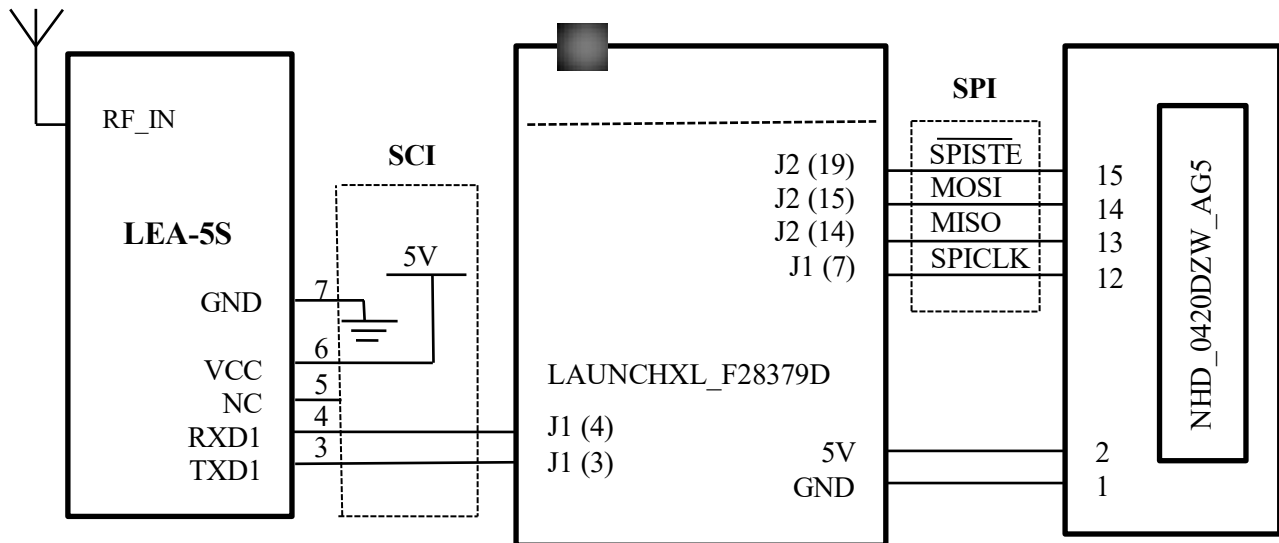
Display technologies also play an important role in embedded clock systems. Earlier designs mainly used LCD displays, which consume more power and offer limited contrast. Recent studies highlight the use of OLED displays due to their low power consumption, compact size, faster response time, and better readability. OLEDs have become a preferred choice for portable and low-power embedded applications.

From the surveyed literature, it is evident that integrating GPS technology with modern microcontroller platforms and OLED displays provides an efficient and accurate timekeeping solution. However, many existing systems are either complex or expensive. This project aims to develop a simple, low-cost, and reliable GPS clock system using a LaunchXL development board and OLED display, addressing the limitations identified.

CHAPTER-3

HARDWARE DESCRIPTION

BLOCK DIAGRAM



1. BLOCK DIAGRAM OF GPS CLOCK SYSTEM USING GPS MODULE

3.1 BLOCK DIAGRAM DISCRIPTION:

The block diagram represents a **GPS-based clock/display system** implemented using a **LEA-5S GPS module**, **LAUNCHXL-F28379D microcontroller**, and an **OLED display**.

1. GPS Antenna and LEA-5S GPS Module

- The GPS antenna receives satellite signals and feeds them to the **RF IN** pin of the **LEA-5S GPS module**.
- The LEA-5S processes these RF signals and determines precise **time, latitude, longitude, and other navigation data**.

- The module operates with a **5 V power supply**, and all grounds are connected to a common **GND**.

2. SCI (UART) Communication Interface

The GPS module communicates with the microcontroller through the **SCI (Serial Communication Interface)**:

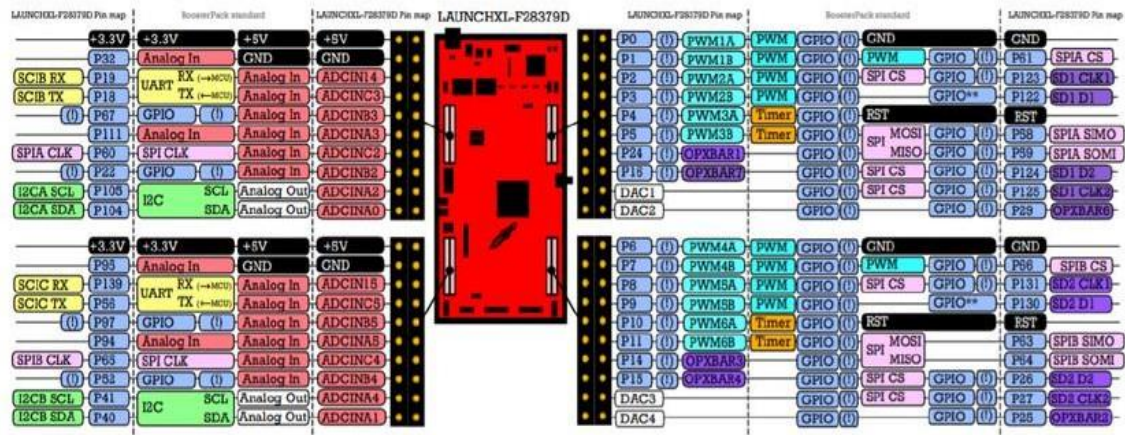
- **TXD1 (GPS) → RX pin of F28379D**
- **RXD1 (GPS) → TX pin of F28379D**

Using SCI, the GPS sends **NMEA sentences** (time and position data) to the microcontroller. This serial communication enables reliable data transfer at a defined baud rate.

3. LAUNCHXL-F28379D Microcontroller

The LAUNCHXL-F28379D acts as the central processing unit of the system. Its main functions are:

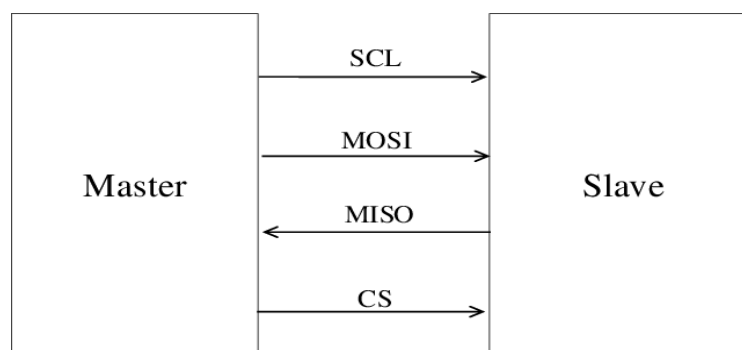
- Receiving GPS data through SCI
- Extracting accurate **time information**
- Formatting the data for display
- Controlling the OLED using SPI
- The microcontroller supplies **5 V and GND** to both the GPS module and the OLED display.



LAUNCHXL PAD (TMS320F28379D) PIN DIAGRAM

The **TMS320F28379D** is a high-performance **C2000** real-time **microcontroller** designed for control applications. It consists of **dual C28x CPU cores** with floating-point support for fast computation. The device includes **Flash, RAM, and Boot ROM** for program and data storage. A **Control Law Accelerator (CLA)** improves real-time control performance. The **analog subsystem** has high-speed **12-bit ADCs** for signal acquisition, while **ePWM modules** generate precise control signals. Multiple **communication interfaces** such as **SCI, SPI, I²C, and CAN** enable external connectivity. Timers, GPIOs, clock, and interrupt systems support reliable real-time operation.

4. SPI Communication Interface



2. PIN DIAGRAM OF SPI

The processed data is sent from the microcontroller to the OLED display using **SPI (Serial Peripheral Interface)**.

SPI signals include:

- **MOSI** (Master Out Slave In)
- **MISO** (Master In Slave Out)
- **SPICLK** (Clock)
- **SPISTE / Chip Select**

SPI is chosen because it provides **high-speed, reliable communication** for display applications.

5. OLED Display (NHD-0420DZW-AG5)

In this system, an OLED display is interfaced with a u-blox GPS module through a microcontroller to display real-time time, date, and location information.

The u-blox GPS module receives signals from multiple satellites and processes them to generate positioning and timing data. This information is output in the form of NMEA sentences (such as \$GPRMC, \$GPGGA) through UART serial communication.

The microcontroller continuously reads this serial data, detects the start of a valid NMEA sentence, and stores it in a buffer. It then checks the sentence type. When a GPRMC sentence is identified, the controller extracts UTC time and date by counting comma delimiters. The UTC time can be converted into IST if required.

The extracted and formatted data is then sent to the OLED display. The OLED is typically interfaced using I²C (or SPI), which requires fewer pins and provides reliable communication. The microcontroller acts as a bridge between the GPS module and the OLED.

Finally, the OLED displays information such as:

- Current time
- Date
- GPS fix status
- Latitude and longitude (optional)

This setup is commonly used in GPS clock systems, tracking devices, and embedded navigation projects because OLED displays are low-power, compact, and provide clear visibility.

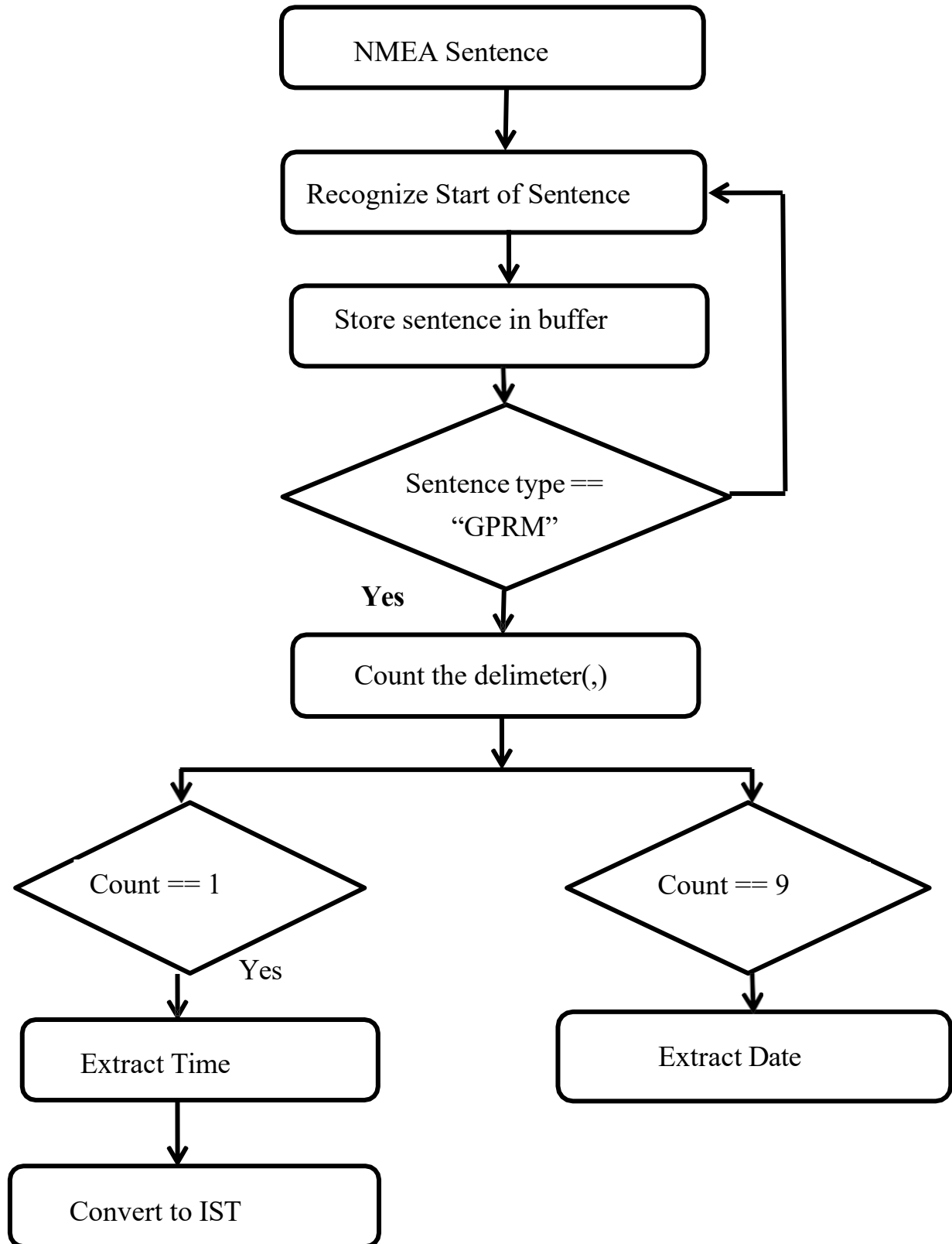


3. OLED DISPLAY



4. U-blox

3.2 FLOW CHART:



This flowchart explains the process of extracting time and date from a GPS NMEA sentence, specifically the GPRMC sentence.

The process starts when an NMEA sentence is received from the GPS module through serial communication. First, the system recognizes the start of a valid NMEA sentence, usually identified by the „\$“ character. Once detected, the entire sentence is stored in a buffer for further processing.

Next, the system checks the sentence type. If the sentence is not GPRMC, it is ignored and the system waits for the next sentence. If the sentence type matches “GPRMC”, the process continues because GPRMC contains both time and date information.

After confirming the sentence type, the system counts the delimiters (commas) present in the sentence. Each comma separates different data fields in the GPRMC format. When the comma count reaches 1, the system identifies the UTC time field and extracts the time. This extracted UTC time is then converted to IST (Indian Standard Time) by adding the appropriate time offset.

Similarly, when the comma count reaches 9, the system identifies the date field in the GPRMC sentence and extracts the date.

Thus, this flow ensures that only valid GPRMC sentences are processed and accurately extracts time and date information from the GPS data stream in an organized and reliable manner.

CHAPTER-4
SOFTWARE DESCRIPTION

4.1 SYSTEM REQUIREMENTS:

1. Hardware Requirements

- **GPS Module**

Used to receive satellite signals and provide accurate time and date information.

- **Microcontroller / DSP (TI LaunchPad)**

Acts as the main controller to process GPS data and control the display.

- **GPS Antenna**

Receives weak satellite signals and feeds them to the GPS module.

- **OLED Display**

Displays the time, date, and GPS status information.

- **Power Supply (5V / 3.3V)**

Provides regulated power to all system components.

- **USB Cable / JTAG Interface**

Used for programming and debugging the microcontroller.

- **Connecting Wires and Breadboard / PCB**

Used for hardware interconnections.

2. Software Requirements

- **Code Composer Studio (CCS)**

Used to write, compile, program, and debug the embedded C code.

- **Embedded C Language**

Used for developing firmware for the microcontroller.

- **TI Device Drivers / Libraries**

Required for peripheral configuration such as UART and I²C.

- **GPS NMEA Protocol Support**

Used to decode GPS data like time and date from NMEA sentences.

- **Host PC (Windows OS)**

Required to run Code Composer Studio and program the controller.

4.2 SYSTEM ENVIRONMENT:

CODE COMPOSER STUDIO:

Code Composer Studio (CCS) is an Integrated Development Environment (IDE) provided by Texas Instruments and is used for the development of embedded software in the GPS Clock System. CCS plays a key role in programming, debugging, and validating the microcontroller used in the system.

In the GPS clock system, CCS is used to write embedded C programs that configure the microcontroller peripherals such as UART/SCI for communication with the GPS module and I²C/SPI for interfacing the OLED display. The software developed in CCS receives NMEA sentences transmitted by the GPS module, processes the data to extract accurate time and date information, and updates the display accordingly.

CCS compiles the written source code using TI's compiler and converts it into machine-level instructions. The generated executable file is then loaded into the microcontroller's memory using the onboard debugger

through JTAG or USB interface.

One of the major advantages of CCS is its powerful debugging capability. It allows real-time monitoring of variables, setting breakpoints, and checking register values. This helps in verifying proper GPS data reception, detecting communication errors, and ensuring correct timing synchronization.

Overall, Code Composer Studio is an essential tool for software development, testing, and performance verification of the GPS clock system.

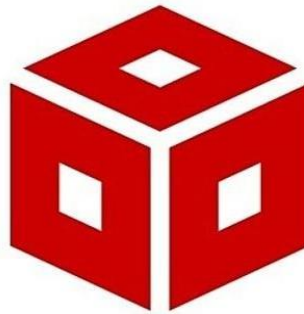


Fig:4.1 code composer studio

4.3 Embedded C Language:

Embedded C is an extension of the C programming language that is widely used for developing software for embedded systems. It is specifically designed to interact directly with hardware components such as microcontrollers, sensors, communication modules, and displays.

In the GPS Clock System, Embedded C is used to develop the firmware that runs on the microcontroller. The program configures on-chip peripherals such as UART/SCI for communication with the GPS module and I²C or SPI for interfacing with the OLED display. It also handles tasks such as receiving GPS NMEA sentences, parsing time and date

information, and updating the display in real time.

Embedded C allows direct access to hardware registers, enabling precise control over timing, memory, and peripheral operations. It supports efficient execution, low memory usage, and real-time performance, which are essential requirements in embedded applications like GPS-based clock systems.

Due to its portability, reliability, and support for low-level hardware interaction, Embedded C is preferred for developing embedded applications. In this project, Embedded C ensures accurate GPS data processing, reliable time synchronization, and stable system operation.

4.4 Advantages of Embedded C:

1. Hardware-Level Control

Embedded C allows direct access to hardware registers and peripherals, enabling precise control of microcontrollers and external devices.

2. High Execution Speed

Programs written in Embedded C execute faster, which is essential for real-time applications such as GPS clock systems.

3. Efficient Memory Usage

Embedded C produces compact code, making it suitable for systems with limited RAM and Flash memory.

4. Portability

Embedded C programs can be easily ported to different microcontrollers with minimal changes.

5. Real-Time Performance

It supports deterministic and predictable execution, which is important for time-critical applications.

6. Better Reliability and Stability

Embedded C provides fine control over system resources, improving system stability and reliability.

7. Wide Microcontroller Support

Almost all microcontroller manufacturers provide Embedded C compilers and libraries.

8. Easy Debugging and Maintenance

Embedded C works well with IDEs like Code Composer Studio, allowing effective debugging and code maintenance.

9. Low Power Consumption

Optimized Embedded C code helps reduce power usage, making it ideal for embedded and battery-powered systems.

10. Industry Standard Language

Embedded C is widely used in industry, making it a standard choice for embedded system development.

4.5 Features of Embedded C:

1. Direct Hardware Access

Allows direct manipulation of hardware registers and peripheral devices.

2. Efficient Use of Memory

Generates compact code suitable for systems with limited RAM

and Flash memory.

3. High Execution Speed

Provides fast program execution, which is essential for real-time embedded applications.

4. Real-Time Capability

Supports deterministic behavior required for time-critical systems.

5. Bit-Level Operations

Supports bitwise operators for controlling individual hardware bits.

6. Interrupt Handling Support

Enables efficient handling of hardware interrupts.

7. Low-Level Programming Support

Allows close interaction with microcontroller architecture.

8. Portability

Code can be reused across different microcontrollers with minimal modifications.

9. Support for Fixed-Point Arithmetic

Useful for systems without floating-point hardware.

10. Small Runtime Environment

Does not require large runtime libraries, reducing overhead.

11. Ease of Integration with IDEs

Works efficiently with tools like Code Composer Studio for development and debugging.

12. Scalability

Suitable for both small and complex embedded applications.

4.6 DOCKLIGHT:

Docklight is a serial communication test and analysis tool used to monitor, analyze, and debug data transmitted between the GPS module and the microcontroller in the GPS Clock System.

In the GPS clock system, the GPS module communicates with the microcontroller using a UART/SCI interface. Docklight is connected to the system through a USB-to-Serial interface to observe the serial data exchanged during operation.

Docklight is mainly used to monitor NMEA sentences transmitted by the GPS module, such as \$GPRMC, \$GPGGA, and \$GPVTG. By observing these messages, developers can verify whether the GPS module is properly receiving satellite signals and transmitting correct time and date information.

Docklight also helps in debugging communication issues. It allows users to check baud rate settings, data format, and data integrity. If incorrect or corrupted data is received, Docklight helps identify whether the issue lies in the GPS module, communication settings, or embedded software.

Overall, Docklight plays an important role in testing, validation, and debugging of serial communication in the GPS Clock System, ensuring reliable GPS data reception and accurate time display.

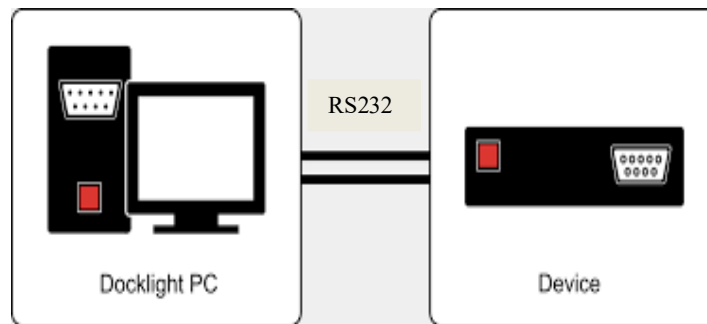


Fig: 4.2 Docklight

4.7 NMEA SENTENCE FORMAT:

NMEA stands for National Marine Electronics Association.

NMEA sentences are text-based messages used by GPS modules to send information like:

- Location (Latitude & Longitude)
- Time
- Speed
- Number of satellites
- Altitude
- Position accuracy
- Direction

These sentences are sent using serial communication (like UART/SPI/I²C depending on module).

Type	Meaning
GGA	Fix data (time, lat, lon, fix quality, satellites)
RMC	Recommended minimum data (date, time, position, speed)
GSA	Satellite status
GSV	Satellites in view
VTG	Course and speed

Table 1: NMEA Sentence Format

- **NMEA Sentence Parts**

1. **\$**

Indicates the **start** of sentence.

2. **Talker ID**

Tells which system is sending data:

Talker ID	Meaning
GP	GPS
GL	GLONASS
GN	GNSS (combined)
GA	Galileo
BD	BeiDou

Table 2: Talker ID

4.8 GPS CLOCK SYSTEM CODE:

```
#include "F2837xD_device.h"
#include "F2837xD_Examples.h"
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

//Macros

#define NMEA_LENGTH 83
#define DATE_LENGTH 7
#define TIME_LENGTH 9
#define OLED_CMD 0x0000
#define OLED_DATA 0x0002
#define OLED_DATA_L1 0x0080
#define OLED_DATA_L2 0x0000
#define OLED_DATA_L3 0x0094
#define OLED_DATA_L4 0x00D4

//Global Variables

unsigned int i;
unsigned int count;
unsigned int nmeaIndex;
char recdata;
char rec [NMEA_LENGTH]
char time [TIME_LENGTH]
char date [DATE_LENGTH]
char fdate[15]= "";
char ftime [15]
char ftime1 [15]
```

```
char ftime2[15]
char ftime3 [15]
char display [TIME_LENGTH]= "";
char display1[15]
char display2[15]
char display3 [15]
char time1 [TIME_LENGTH]
struct Ist time
{
unsigned int hrs;
unsigned int min;
unsigned int sec;
} Ist_time, Jst_time, Est_time, UK_time;
#include "F2837xD_device.h"
#include "F2837xD_Examples.h"
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
//Macros #define NMEA_LENGTH 83 #define DATE_LENGTH 7
#define TIME_LENGTH 9
#define OLED_CMD 0x0000
#define OLED_DATA 0x0002
#define OLED_DATA L1 0x0080
#define OLED_DATA L2 0x0000
#define OLED_DATA L3 0x0094
#define OLED_DATA L4 0x0004
//Global Variables
unsigned int 1;
```

```
unsigned int count;
unsigned int nmeaIndex;
char recdata;
char rec[NMEA_LENGTH] == " ";
char time [TIME_LENGTH] == " ";
char date [DATE_LENGTH] == " ";
char fdate [15] = " ";
char ftime [15] = " ";
char ftime1[15] = " ";
char ftime2[15] = " ";
char ftime3[15] = " ";
char display [TIME_LENGTH] = " ";
char display1[15] = " ";
char display2[15] = " ";
char time1 [TIME_LENGTH] = " ";
struct Ist_time {
    unsigned int hrs;
    unsigned int min;
    unsigned int sec;
}Ist_time, Ist_time, Est_time, UK_time;
void Sci_Gpio_Init(void);
void Sci_Init(void);
    interrupt void SciRx_isr(void);
void Spi_Gpio_Init(void);
void Spi Init(void);
void Time_ist(void);
void nmea_parse(const char sentence[]);
void oledinit(void);
```

```
void writedata(char);
void OLED_Display_data(char *str);
void writecmd(char);
void oled(char *ftime, char *ftime1, char *ftime2, char *ftime3);
void convert(char *date, char *time);
void main(void)
{
    InitSysCtrl();
    InitGpio();
    DINT;
    InitPieCtrl();
    IER = 0x0000;
    IFR = 0x0000;
    InitPieVectTable();
    Sci_Gpio_Init();
    Spi_Gpio_Init();
    Sci_Init();
    Spi_Init();
    oledinit();
    while(1)
    {
    }
}

void Sci_Init(void)
{
    EALLOW;
    CpuSysRegs.CPUSYSLOCK1.bit.PCLKCR7 = 1;
    CpuSysRegs.PCLKCR7.bit.SCI_B = 1;
```

```
EDIS;
ScibRegs.SCICTL1.bit.SWRESET=0;
ScibRegs.SCICTL1.all
=0;
ScibRegs.SCICCR.all=0x0007;
ScibRegs.SCIHBAUD.all=0X0002;
// txena, rxena
// char length 8
// 9600
ScibRegs.SCILBAUD.all=0X008A;
ScibRegs.SCICTL2.bit.RXBKINTENA
= 1;
ScibRegs.SCICTL1.bit.SWRESET
= 1;
// software reset
PieCtrlRegs.PIECTRL.bit.ENPIE
= 1;
PieCtrlRegs.PIEIER9.bit.INTx3
= 1;
EALLOW;
PieVectTable.SCIB_RX_INT
&SciRx_isr;
EDIS;
IER |=M_INT9;
EINT;
ERTM;
void Spi_Init(void)
{
```

```
SpiaRegs.SPICCR.all 0x000F;
SpiaRegs.SPICTL.all = 0x0007;
SpiaRegs.SPIBRR.bit.SPI_BIT_RATE= 0x7F;
SpiaRegs. SPICCR.bit.SPISWRESET = 1;
}
interrupt void SciRx_isr(void)
{
count++;
recdata ScibRegs.SCIRXBUF.all;
if (recdata == '$')
{
{
if(rec[3] == 'R')
nmea_parse(rec);
Time_ist();
// UTC to IST
oledinit();
convert(date, time);
oled (ftime, ftime1, ftime2, ftime3);
writecmd (OLED_DATA_L1);
OLED_Display_data(display);
writecmd(OLED_DATA_L2);
OLED_Display_data(display1);
writecmd (OLED DATA_L3);
OLED Display data(display2);
OLED_Display_data(display2);
writecmd(OLED_DATA_L4);
OLED_Display_data(display3);
```

```
}  
count = 0;  
nmeaIndex = 0;  
}  
{  
}  
if(nmeaIndex < 83)  
rec[nmeaIndex++] = recdata;  
PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;  
}  
void Sci_Gpio_Init(void)  
{  
EALLOW;  
GpioCtrlRegs.GPAGMUX2.bit.GP1018 = 0; //SCI B TX  
GpioCtrlRegs.GPAMUX2.bit.GP1018 = 2;  
GpioCtrlRegs.GPADIR.bit.GPIO18 = 1;  
GpioCtrlRegs.GPAPUD.bit.GPIO18 = 0;  
GpioCtrlRegs.GPAGMUX2.bit.GP1019 = 0;  
GpioCtrlRegs.GPAMUX2.bit.GP1019 = 2;  
GpioCtrlRegs.GPADIR.bit.GPIO19 = 0;  
GpioCtrlRegs.GPAPUD.bit.GPIO19 = 0;  
EDIS;  
}  
//SCI B RX  
void Spi_Gpio_Init(void)  
{  
EALLOW;  
GpioCtrlRegs.GPBGMUX2.bit.GP1058=3;  
GpioCtrlRegs.GPBMUX2.bit.GP1058=3;
```

```
GpioCtrlRegs.GPBDIR.bit.GP1058=1; //SIMO
GpioCtrlRegs.GPBPUD.bit.GPIO58=0;
GpioCtrlRegs.GPBMUX2.bit.GPIO59=3;
GpioCtrlRegs.GPBGMUX2.bit.GPIO59=3; //SOMI
GpioCtrlRegs.GPBDIR.bit.GPIO59=0;
GpioCtrlRegs.GPBPUD.bit.GPIO59=0;
GpioCtrlRegs.GPBGMUX2.bit.GP1060=3; //CLK
GpioCtrlRegs.GPBMUX2.bit.GPIO60=3;
GpioCtrlRegs.GPBPUD.bit.GP1060=0;
GpioCtrlRegs.GPBGMUX2.bit.GP1061=3;
GpioCtrlRegs.GPBMUX2.bit.GP1061=3;
GpioCtrlRegs.GPBPUD.bit.GP1061=0;
//CS
EDIS;
}
void nmea_parse(const char sentence[])
{
    unsigned int i
    = 0;
    unsigned int j
    = 0;
    unsigned int time_len
    0;
    unsigned int date_len
    = 0;
    unsigned int commacount
    = 0;
    unsigned int comm countindex
    = 0;
```

```
for(j= 7; sentence[j] != ',' && sentence[j] !='\0'; j++) //?
{
}
time[time_len++] sentence[j];
time[time_len] = '\0';
for(j = 0; sentence[j] != '\0'; j++)
{
if (sentence[j] == ',')
{
commacount++;
}
{
if(commacount == 9)
comm_countindex = j;
for(i = comm_countindex+1; i < comm_countindex + 7; i++)
{
date[date_len++] sentence[i];
}
break;
}
}
}
void Time_ist(void)
{
sscanf(time,      "%2d%2d%2d",      &Ist_time.hrs,      &Ist_time.min,
&Ist_time.sec);
sscanf(time,      "%2d%2d%2d",      &Jst_time.hrs,      &Jst_time.min,
&Jst_time.sec);  sscanf(time,      "%2d%2d%2d",      &Est_time.hrs,
&Est_time.min, &Est_time.sec);
```

```
sscanf(time, "%2d%2d%2d", &UK_time.hrs, &UK_time.min,
&UK_time.sec);
Ist_time.hrs += 5; Ist_time.min += 30; //INDIAN
Jst_time.hrs += 9;
Jst_time.min += 0; //JAPAN
Est_time.hrs=4;
Est_time.min += 0; //NEWYORK
UK_time.hrs += 1;
UK_time.min += 0; //UK
while(Ist_time.min >=60)
{
Ist_time.min-= 60;
Ist_time.hrs++;
} while(Ist_time.hrs > -24)
{
}
Ist_time.hrs-=24;
ftime[0]= (Ist_time.hrs/10)+'0';
ftime[1]= (Ist_time.hrs%10)+'0';
ftime[2]= ':';
ftime[3]= (Ist_time.min/10)+'0';
ftime[4]= (Ist_time.min%10)+'0';
ftime[5]= ':';
ftime[6]= (Ist_time.sec/10)+'0';
ftime [7]= (Ist_time.sec%10)+'0';
ftime [8]= '\0';
ftime1[0]= (Jst_time.hrs/10)+'0';
```

```
ftime1[1]=(Jst_time.hrs%10)+'0';
ftime1[2]=":'";
ftime1[3]=(Ist_time.min/10)+'0';
ftime1[4]=(Ist_time.min%10)+'0';
ftime1[5]=":'";
ftime1[6]=(Ist_time.sec/10)+'0';
ftime1[7]=(Ist_time.sec%10)+'0';
ftime1[8]='\\0';
ftime2[0]=(Est_time.hrs/10)+'0';
ftime2[1]=(Est_time.hrs%10)+'0';
ftime2[2]=":'";
ftime2[3]=(Est_time.min/10)+'0';
ftime2[4]=(Est_time.min%10)+'0';
ftime2[5]=":'";
ftime2[6]=(Est_time.sec/10)+'0';
ftime2[7]=(Est_time.sec%10)+'0';
time2[8]='\\0';
ftime3[0]=(UK_time.hrs/10)+'0';
ftime3[1]=(UK_time.hrs%10)+'0';
time3[2]=":'";
ftime3[3]=(UK_time.min/10)+'0';
ftime3[4]=(UK_time.min%10)+'0';
time3[5]=":'";
ftime3[6]=(UK_time.sec/10)+'0';
ftime3[7]=(UK_time.sec%10)+'0';
ftime3[8]='\\0';
void oled_init(void) {
    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
    writecmd(0x38); //function set (DL 8bit, English)
```

```
DELAY_US(2000);
writecmd(0x08);
//display off
DELAY_US(1000);
writecmd(0x01);
//display clear
DELAY_US (3000);
writecmd(0x06);
//entry mode set (cursor auto increment)
DELAY_US(1000);
//home command (cursor initial position)
writecmd(0x02);
DELAY_US(1000);
writecmd(0x0F); //display on
DELAY_US(1000);
GpioDataRegs.GPBSET.bit.GPIO61=1; //SET CS
EDIS;
}
void writecmd(char cmd)
{
EALLOW;
GpioDataRegs.GPBCLEAR.bit.GP1061=1; //clear CS
GpioDataRegs.GPBCLEAR.bit.GPIO611; //clear CS
while(SpiaRegs.SPISTS.bit、 BUFFULL_FLAG == 1);
SpiaRegs.SPITXBUF ((OLED_CMD <<8) | cmd) << 6;
DELAY_US (1000);
GpioDataRegs.GPBSET.bit.GPIO61= 1; //SET CS
EDIS;
}
```

```
void writedata(char data)
{
    EALLOW;
    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1; //clear CS
    while(SpiaRegs.SPISTS.bit.BUFFULL_FLAG == 1);
    SpiaRegs.SPITXBUF ((OLED_DATA << 8) | data) << 6;
    DELAY_US(1000);
    GpioDataRegs.GPBSET.bit.GPIO61; //SET CS
    EDIS;
}

void OLED_Display_data(char *str)
{
    while(*str)
    {
        writedata(*str++);
    }
}

void convert(char *date, char *time)
{
    Time_ist();
    //ftime[0]= time[0];
    //ftime[1]= time[1];
    //ftime [2]=':.';
    //ftime[3]=time[2];
    //ftime[4]= time[3];
    //ftime [5]=':.';
    //ftime[6]=time[4];
    //ftime[7]=time[5];
}
```

```
//ftime[8]= '\0';
fdate[0]= date[0];
fdate[1]= date[1];
fdate[2]= '/';
fdate[3]= date[2];
fdate[4]= date[3];
fdate [5] = '/';
fdate [6]= '2';
fdate [7]= '0';
fdate[8]= date[4];
fdate[9]= date[5];
fdate[10]= '\0';
}

void oled(char *ftime, char *ftime1, char *ftime2, char *ftime3)
{
strcpy(display, "IND: ");
strcat(display, ftime);
strcpy(display1, "JAP:");
strcat(display1, ftime1);
strcpy(display2, "NYK:");
strcat(display2, ftime2);
strcpy(display3, "UK: ");
strcat(display3, ftime3);
}
```

CHAPTER-5

ADVANTAGES AND APPLICATIONS

5.1 ADVANTAGES:

1. High Accuracy:

GPS clock systems provide extremely accurate time (nanosecond-level accuracy) because they use atomic clocks from GPS satellites.

2. Global Time Reference:

The same time reference is available anywhere in the world, making it ideal for global and distributed systems.

3. No Manual Time Setting:

Time is automatically obtained from GPS satellites, eliminating human error in setting or correcting time.

4. Automatic Time Synchronization:

The clock continuously synchronizes with satellite signals, preventing time drift.

5. Independent of Internet:

GPS clocks do not require internet connectivity, unlike NTP-based systems.

6. Supports Multiple Time Zones:

A single GPS UTC time can be converted into different local time zones (IST, UTC, EST, etc.).

7. High Reliability:

GPS satellites operate continuously, providing stable and reliable timing information.

8. Useful for Embedded Systems:

GPS clock systems are well-suited for embedded applications such as microcontrollers, DSPs, and real-time systems.

9. Low Power Consumption:

Modern GPS modules consume low power, making them suitable for portable and battery-operated devices.

10. Wide Application Usage:

Used in telecom networks, power grids, navigation systems, financial transactions, and scientific research.

5.2 APPLICATIONS:

1. Telecommunication Networks:

Used for precise time synchronization in mobile networks (4G/5G), base stations, and data transmission systems.

2. Power Grid Synchronization:

GPS clocks are used in substations for fault detection, event recording, and phase synchronization in smart grids.

3. Navigation Systems:

Provides accurate timing for GPS-based navigation in vehicles, ships, aircraft, and drones.

4. Financial Systems:

Used in stock exchanges and banking systems to timestamp transactions accurately.

5. Embedded and Real-Time Systems:

Provides precise timing for microcontroller and DSP-based systems requiring accurate time stamping.

6. Scientific Research:

Used in astronomy, geophysics, and space research where precise time measurement is essential.

7. Broadcasting Systems:

Ensures synchronized audio and video transmission in radio and television broadcasting.

8. Defense and Military Applications:

Used for secure communication, navigation, and synchronized military operations.

9. Traffic Management Systems:

Helps in synchronized traffic signals, vehicle tracking, and intelligent transportation systems.

10. Data Centers and Servers:

Used for accurate log time-stamping and synchronization of distributed servers.

CHAPTER-6

RESULTS

ted into a readable form and displayed on the OLED screen using SPI communication.

The OLED display showed clear and stable time information across multiple lines, updating continuously in real time. There was no visible flickering or delay during operation, demonstrating efficient synchronization between GPS data reception, processing, and display. The interrupt-based SPI reception mechanism ensured that incoming GPS data was handled without loss, even during continuous data streaming.

The system demonstrated high accuracy, as the displayed time closely matched standard reference clocks. Since the timing information was obtained directly from GPS satellites equipped with atomic clocks, the system showed negligible time drift during prolonged operation. This makes the GPS Clock System highly reliable for applications requiring precise time synchronization.

6.2 CONCLUSION:

Overall, the experimental results confirm that the developed GPS Clock System is accurate, stable, and efficient. The successful integration of GPS, microcontroller, and OLED display validates the system's suitability for real-time embedded applications such as communication systems, power grid synchronization, and scientific instrumentation.

CHAPTER-7

REFERENCES

7.1 REFERENCES:

1. Kaplan, E. D., & Hegarty, C. J., *Understanding GPS: Principles and Applications*, Artech House, 2nd Edition, 2006.
2. Mira, P., & Enge, P., *Global Positioning System: Signals, Measurements, and Performance*, Ganga-Jamuna Press, 2nd Edition, 2011.
3. Texas Instruments, *LaunchXL Development Board Technical Reference Manual*, Texas Instruments Incorporated.
4. u-blox AG,
GPS Receiver Protocol Specification – NMEA, u-blox Documentation.
5. Tom Igoe, *Making Things Talk: Using Sensors, Networks, and Arduino to See, Hear, and Feel Your World*, O'Reilly Media, 2011.
6. Datasheet, *OLED Display Module (SSD1306 Controller)*, Solomon Systech Limited.
7. Jan Axelson, *Serial Port Complete: Programming and Circuits for RS-232 and RS-485*, Lakeview Research, 2nd Edition.
8. National Marine Electronics Association (NMEA), *NMEA 0183 Standard*, Official Specification Document.